Great Place To Work. Certified FEB 2023 – FEB 2024 INDIA

UPES

UNIVERSITY OF TOMORROW

nirf RANKING 2025

*RANKED 45 IN INDIA

#University Category

QS WORLD UNIVERSITY RANKINGS

NO.1 PVT. UNIVERSITY IN ACADEMIC REPUTATION IN INDIA

NAAC

ACCREDITED **GRADE 'A'** BY NAAC

PERFECT SCORE OF **150/150** AS A TESTAMENT TO EXECEPTIONAL E-LEARNING METHODS

UPES UNIVERSITY OF TOMORROW

Applied Machine Learning

Unit 06 – Lecture 13: Classification Foundations and K-Nearest Neighbour

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

The first classifier, and the one you can check with a pen

Topic focus: **k-NN has no training step at all**

Repository: `https://github.com/tali7c/Applied-Machine-Learning`

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.;James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.;Zhang et al., *Dive into Deep Learning*, Ch. 5.

Quick Links

The setting | Instance-based | k-NN | Choosing k | Scaling | Metrics | The curse | Failure modes | Summary

Agenda

The classification setting

Instance-based learning, measured

K-nearest neighbour

Choosing k

Scaling is not optional

Distance metrics and weighted voting

The curse of dimensionality

Ties, imbalance, and the rest of the failure list

Summary

How this lecture works

Every idea appears twice

1. **By hand** — seven points, one query, two patients, five neighbours. Small enough to check on paper, and small enough to appear in a test.
2. **In code** — the same arithmetic on real data, so you see that `KNeighborsClassifier` does exactly that and nothing magic.

Commit before you look

Five slides ask you to predict a number before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

Two numbers to carry out: $k = 1$ **scores exactly** 1.000000 **on its own training set** and is not a good model; and k-NN on `wine` scores 0.6633 **unscaled and** 0.9608 **scaled**.

Four datasets, one seed — and one thing a seed cannot

The data

scikit-learn's bundled `load_iris`, `load_wine`, `load_breast_cancer` and `load_digits`, plus three synthetic sets each fixed by `make_classification(..., random_state=0)`. Nothing is downloaded.

The randomness

One seed: `np.random.default_rng(0)` and `random_state=0` in every split and shuffle. Every listing that draws or splits shows it.

Except the timings

Milliseconds depend on your processor and no seed fixes them. Every cost claim in this deck is therefore **also** stated as an exact operation count. **Quote the count, never the millisecond.**

Where we are

The regression unit predicted a number

A price, a blood-sugar reading, a temperature. We measured error as a distance along a number line, because that is what the target lived on.

This unit predicts a label

Benign or malignant. Setosa, versicolor or virginica. One of ten digits. **The labels have no arithmetic.** A prediction is right or wrong.

You have already met one classifier

Logistic regression fitted a probability and compared it with a threshold. Today we step back and ask what *all* classifiers have in common — and then forward, to the simplest one there is.

And the preprocessing rule survives intact

A scaler learns a mean from the training rows only. Today that rule stops being hygiene and starts being worth **thirty points of accuracy**.

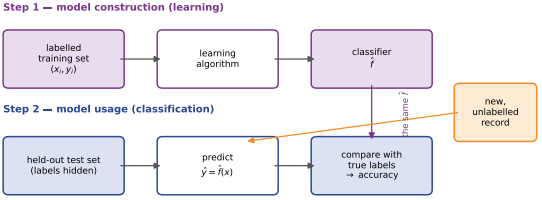
From "how much" to "which"

A classifier is a function from the feature space to a *finite* label set, learned from (features, correct label) examples.

Term	Also called	Meaning
instance	sample, tuple, record, row	one thing to be classified
feature	attribute, predictor, column	one measurement of that thing
feature vector	$x \in \mathbb{R}^d$	all d measurements of one instance
class label	target, response, y	the correct answer for that instance
training set	labelled data	the instances the classifier learns from
test set	held-out data	instances used only to estimate accuracy
classifier	model, hypothesis, $\hat{f}$	the learned label-predicting rule
decision boundary	—	where $\hat{f}$ changes its answer

Han, Kamber and Pei say *tuple* and *attribute*; scikit-learn says *sample* and *feature*. **You will meet both, and the examination uses the first.**

The general approach: two steps



1. Model construction

A learning algorithm is given instances whose labels are *known* and produces a classifier. Labels supplied ⇒ **supervised** learning.

2. Model usage

Estimate accuracy on an *independent* test set; if it is acceptable, label genuinely unknown records.

A model that has seen an instance can recite its label, and reciting is not predicting.
Two sections from now that becomes a proof.

The whole workflow is three lines

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
                                      random_state=0, stratify=y)
clf = KNeighborsClassifier(n_neighbors=3) # step 1: construction
clf.fit(Xtr, ytr)
pred = clf.predict(Xte) # step 2: usage
```

```
training set (105, 4)  test set (45, 4)
test accuracy 1.0000
confusion matrix (rows = true, cols = predicted)
[[15  0  0]
 [ 0 15  0]
 [ 0  0 15]]
```

The whole workflow is three lines

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
                                      random_state=0, stratify=y)
clf = KNeighborsClassifier(n_neighbors=3) # step 1: construction
clf.fit(Xtr, ytr)
pred = clf.predict(Xte) # step 2: usage
```

```
training set (105, 4)  test set (45, 4)
test accuracy 1.0000
confusion matrix (rows = true, cols = predicted)
[[15  0  0]
 [ 0 15  0]
 [ 0  0 15]]
```

Forty-five out of forty-five. Do not be pleased.

A clean sweep on 45 easy test rows is **not a result, it is a small sample** — and a confusion matrix of zeros teaches you nothing about reading one. Score all 150 rows by cross-validation instead.

Do it properly: all 150 rows, cross-validated

```
CV = StratifiedKFold(5, shuffle=True, random_state=0) # the one seed
cvp = cross_val_predict(KNeighborsClassifier(3), X, y, cv=CV)
```

```
accuracy 0.9600
confusion matrix (rows = true, cols = predicted)
[[50  0  0]
 [ 0 47  3]
 [ 0  3 47]]
```

Do it properly: all 150 rows, cross-validated

```
CV = StratifiedKFold(5, shuffle=True, random_state=0) # the one seed
cvp = cross_val_predict(KNeighborsClassifier(3), X, y, cv=CV)
```

```
accuracy 0.9600
confusion matrix (rows = true, cols = predicted)
[[50  0  0]
 [ 0 47  3]
 [ 0  3 47]]
```

Read the matrix, not the number

Every *setosa* found — it is linearly separable from the rest. **All six errors are between versicolor and virginica**, three each way. The single number 0.9600 hides *which* mistake was made — and in a medical or a credit application, the identity of the mistake is the whole question.

Precision and recall, from that same matrix

	precision	recall	f1-score	support
setosa	1.000	1.000	1.000	50
versicolor	0.940	0.940	0.940	50
virginica	0.940	0.940	0.940	50
accuracy			0.960	150
macro avg	0.960	0.960	0.960	150
weighted avg	0.960	0.960	0.960	150

Precision and recall, from that same matrix

	precision	recall	f1-score	support
setosa	1.000	1.000	1.000	50
versicolor	0.940	0.940	0.940	50
virginica	0.940	0.940	0.940	50
accuracy			0.960	150
macro avg	0.960	0.960	0.960	150
weighted avg	0.960	0.960	0.960	150

Recall of *versicolor* = 47/50

Of the flowers that really *were* versicolor, we found 94%. Read it *along the row*.

Precision of *versicolor* = 47/50

Of the flowers we *called* versicolor, 94% really were. Read it *down the column*. Equal here only because the errors balance.

A later lecture in this unit builds ROC and AUC on top of this. **Until then: always show the matrix as well as the number.**

The families of classification algorithm

Family	Example	How it draws the boundary
instance-based	k-nearest neighbour	the training points themselves define it; no formula *(this lecture)*
linear	logistic regression, linear SVM	one hyperplane $w^\top x + b = 0$
tree-based	ID3, CART	recursive axis-parallel splits
probabilistic	naïve Bayes	where the posterior probabilities are equal
kernel	SVM with an RBF kernel	a hyperplane in a transformed space
ensemble	bagging, boosting, random forest	many boundaries, combined by vote
neural	multi-layer perceptron	composed non-linear maps

The feature space is $\mathbb{R}^d$; a classifier cuts it into regions, one label per region. **Every algorithm in this unit is a different way of drawing the surfaces between those regions.**

Predict first #1

`load_breast_cancer`: 569 patients, 30 features, five-fold cross-validation inside a `Pipeline` with `StandardScaler`. Six classifiers, one of which always answers with the commonest class.

How far apart are the five real classifiers? And where does the baseline sit?

Predict first #1

`load_breast_cancer`: 569 patients, 30 features, five-fold cross-validation inside a `Pipeline` with `StandardScaler`. Six classifiers, one of which always answers with the commonest class.

How far apart are the five real classifiers? And where does the baseline sit?

classifier	CV acc	std
KNeighborsClassifier(5)	0.9649	0.0215
LogisticRegression	0.9789	0.0142
GaussianNB	0.9297	0.0124
DecisionTreeClassifier	0.9262	0.0218
SVC(rbf)	0.9771	0.0070
DummyClassifier	0.6274	0.0039

Predict first #1

`load_breast_cancer`: 569 patients, 30 features, five-fold cross-validation inside a `Pipeline` with `StandardScaler`. Six classifiers, one of which always answers with the commonest class.

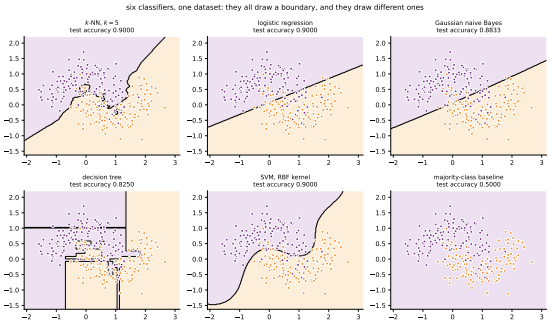
How far apart are the five real classifiers? And where does the baseline sit?

classifier	CV acc	std
KNeighborsClassifier(5)	0.9649	0.0215
LogisticRegression	0.9789	0.0142
GaussianNB	0.9297	0.0124
DecisionTreeClassifier	0.9262	0.0218
SVC(rbf)	0.9771	0.0070
DummyClassifier	0.6274	0.0039

The spread between the five is smaller than the gap to the baseline

Best to worst of the real classifiers: 0.9789 to 0.9262, about five points. Worst real classifier to `DummyClassifier`: **thirty points**. **Choosing the algorithm is usually a smaller decision than people expect; choosing the features is usually a larger one.** And note k-NN is a full point behind logistic regression here.

Drawn: six classifiers, one dataset



The same 280 training points. Every one of them partitions the plane; they disagree about *how*. k-NN follows the data, logistic regression insists on a straight line, the tree can only cut parallel to the axes — and the baseline paints the whole plane one colour.

Eager and lazy: the distinction that organises today

An **eager** learner digests the data into a model at training time and then throws the data away. A **lazy** learner stores the data and defers everything until a prediction is demanded.

	Eager	**Lazy**
training cost	high	essentially zero
prediction cost	low	high, and grows with n
memory after training	the model only	the whole training set
what it commits to	one global model	nothing until asked
new training data	usually refit from scratch	append the rows
examples	logistic regression, trees, SVM	k-NN, kernel regression

Lazy learners are also called **instance-based**, **memory-based** or *non-parametric*. **"Fitting" k-NN means keeping the table.**

Cost is cheap to assert. Here it is, measured

`load_digits`: 1257 training rows, 540 test rows, 64 features. Best of five repetitions each.

```
digits: 1257 training rows, 540 test rows, 64 features
model                      fit (ms)   pred (ms)   pred/fit      acc
KNeighborsClassifier(5)       0.234       3.077    13.1455   0.9796
LogisticRegression          844.267       0.101     0.0001   0.9611
DecisionTreeClassifier       10.019       0.079     0.0079   0.8426
GaussianNB                    0.856       0.749     0.8744   0.8481

  KNN  predict: 540 x 1257 distances x 64 features = 43441920 coordinate ops
  KNN  fit    : 0 distances -- it copies 80448 numbers and stops
  LR   predict: 540 x 10 classes x 64 features = 345600 multiply-adds
  LR   fit    : 172 L-BFGS iterations over all 1257 training rows
```

Cost is cheap to assert. Here it is, measured

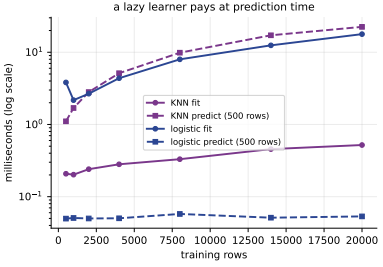
`load_digits`: 1257 training rows, 540 test rows, 64 features. Best of five repetitions each.

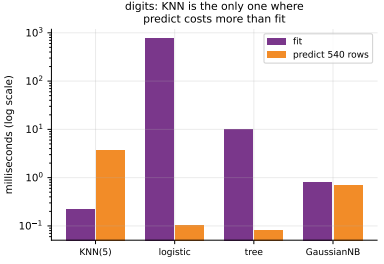
```
digits: 1257 training rows, 540 test rows, 64 features
model                        fit (ms)   pred (ms)   pred/fit      acc
KNeighborsClassifier(5)         0.234       3.077    13.1455   0.9796
LogisticRegression            844.267       0.101     0.0001   0.9611
DecisionTreeClassifier         10.019       0.079     0.0079   0.8426
GaussianNB                      0.856       0.749     0.8744   0.8481

  KNN  predict: 540 x 1257 distances x 64 features = 43441920 coordinate ops
  KNN  fit    : 0 distances -- it copies 80448 numbers and stops
  LR   predict: 540 x 10 classes x 64 features = 345600 multiply-adds
  LR   fit    : 172 L-BFGS iterations over all 1257 training rows
```

Quote the counts. The milliseconds are this machine only.

Per prediction k-NN does **125.7**$\times$ the arithmetic logistic regression does — exact, on any machine. Its `fit` does *no* distance computations at all. The `pred/fit` column says the same thing with a clock on it: above one for k-NN, 0.0001 for logistic regression, a factor of about 10^5 apart. **Timings move between runs on the same machine — this ratio measured between 11 and 14. Never read one aloud as a fact.**

Drawn: a lazy learner pays at prediction time





Left: as the training set grows from 500 to 20,000 rows, k-NN's fit time stays flat and its prediction time grows *linearly*, while logistic regression does the opposite. Right: the same contrast on `digits`, on a log scale.

Why: $O(nmd)$, and nothing you can do about it

`fit` does one thing

It copies the training matrix. `predict` must, for every one of m query rows, compute a distance to every one of n stored rows over d features: $O(nmd)$.

So doubling the data doubles the latency

Not the training time — the *latency of every answer you will ever give*. That is a deployment fact, not a benchmark curiosity.

train rows	KNN fit (ms)	KNN predict (ms)	LR fit (ms)	LR pred (ms)	KNN distances
500	0.206	1.016	2.662	0.061	250000
2000	0.237	2.702	2.700	0.049	1000000
8000	0.295	9.921	10.945	0.058	4000000
20000	0.523	22.039	17.430	0.045	10000000

Why: $O(nmd)$, and nothing you can do about it

`fit` does one thing

It copies the training matrix. `predict` must, for every one of m query rows, compute a distance to every one of n stored rows over d features: $O(nmd)$.

So doubling the data doubles the latency

Not the training time — the *latency of every answer you will ever give*. That is a deployment fact, not a benchmark curiosity.

train rows	KNN fit (ms)	KNN predict (ms)	LR fit (ms)	LR pred (ms)	KNN distances
500	0.206	1.016	2.662	0.061	250000
2000	0.237	2.702	2.700	0.049	1000000
8000	0.295	9.921	10.945	0.058	4000000
20000	0.523	22.039	17.430	0.045	10000000

Read the last column. Forty times the training rows is **exactly** forty times the distance computations, 10,000,000 against 250,000 — true on every machine. The measured time grew by less, and by a different factor each run: that gap is BLAS, not mathematics. Logistic regression's prediction time did not move at all.

And it has to keep the data

```
kn = KNeighborsClassifier(5).fit(Xdtr, ydtr)
print(kn._fit_X.shape, kn._fit_X.nbytes)
lr = LogisticRegression(max_iter=5000).fit(Xdtr, ydtr)
print(lr.coef_.nbytes + lr.intercept_.nbytes)
```

```
stored training matrix shape (1257, 64)
bytes of training data held   643584
logistic regression stores    5200 bytes, coef_ shape (10, 64)
ratio 123.8 x
```

And it has to keep the data

```
kn = KNeighborsClassifier(5).fit(Xdtr, ydtr)
print(kn._fit_X.shape, kn._fit_X.nbytes)
lr = LogisticRegression(max_iter=5000).fit(Xdtr, ydtr)
print(lr.coef_.nbytes + lr.intercept_.nbytes)
```

```
stored training matrix shape (1257, 64)
bytes of training data held   643584
logistic regression stores    5200 bytes, coef_ shape (10, 64)
ratio 123.8 x
```

k-NN keeps your training data, and that is a privacy fact

An eager model can be shipped without the data it was trained on. **A *k*-NN model is the data.** If the training rows are patient records or exam scripts, deploying the classifier means deploying those records — and the ratio grows with n while the logistic model's size does not. This is not a technicality: it decides whether a model can leave the building.

Can a KD-tree rescue the prediction cost?

Partly — and only in low dimensions. 5000 points, brute force against a KD-tree, at three dimensionalities.

features	algo	fit (ms)	predict (ms)	leaves opened per query
2	brute	0.314	9.055	
2	kd_tree	1.489	15.361	1.5
10	brute	0.289	10.742	
10	kd_tree	2.767	39.714	28.6
40	brute	0.287	16.707	
40	kd_tree	6.931	169.059	63.8

Can a KD-tree rescue the prediction cost?

Partly — and only in low dimensions. 5000 points, brute force against a KD-tree, at three dimensionalities.

features	algo	fit (ms)	predict (ms)	leaves opened per query
2	brute	0.314	9.055	
2	kd_tree	1.489	15.361	1.5
10	brute	0.289	10.742	
10	kd_tree	2.767	39.714	28.6
40	brute	0.287	16.707	
40	kd_tree	6.931	169.059	63.8

At 40 features the tree prunes almost nothing

Ignore the clock and read the last column, which is exact. At two features a query opens 1.5 leaves and 6249 subtrees are thrown away unlooked-at. At forty features only 127 subtrees are pruned in a thousand queries and each query opens 63.8 leaves — the whole tree. It computes every distance brute force would have *and* pays for the traversal. **Leave `algorithm='auto'` alone; scikit-learn already knows this.**

The entire algorithm

To classify a new instance q: measure the distance from q to every training instance, keep the k closest, and return the commonest label among them.

Given $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, a distance $d(\cdot, \cdot)$ and an integer k, let $N_k(q)$ be the k instances with smallest $d(q, x_i)$. Then

$$\hat{f}(q) = \arg\max_c \sum_{(x_i, y_i) \in N_k(q)} \mathbb{1}[y_i = c],$$

with the default distance $d(p, q) = \sqrt{\sum_j (p_j - q_j)^2}$.

Two examination habits

(1) Ranking by d and by d^2 gives the same order. Work in squared distances and take one square root at the end — or none.

(2) k is *not* learned from the data. It is a **hyperparameter**, chosen by validation.

By hand: seven points, one query

Training set: $A(1,1)$, $B(2,2)$, $C(1,3)$ class 0; $D(5,4)$, $E(6,5)$, $F(4,7)$ class 1; and $G(6,2)$ class 0. Classify $q_1 = (3,3)$.

point	A	B	C	D	E	F	G
$(x_1-3)^2$	4	1	4	4	9	1	9
$(x_2-3)^2$	4	1	0	1	4	16	1
d^2	8	**2**	4	5	13	17	10
d	2.8284	**1.4142**	2.0000	2.2361	3.6056	4.1231	3.1623
class	0	**0**	0	1	1	1	0

Ranked nearest first: **B, C, D, A, G, E, F**. The nearest neighbour is B at $d = \sqrt{2} = 1.4142$, whose class is **0**. **1-NN predicts class 0.** No square roots were needed to get there.

The same thing in code

```
Xh = np.array([[1,1],[2,2],[1,3],[5,4],[6,5],[4,7],[6,2]], dtype=float)
yh = np.array([0, 0, 0, 1, 1, 1, 0])
knn1 = KNeighborsClassifier(n_neighbors=1).fit(Xh, yh)
dist, idx = knn1.kneighbors(np.array([[3.0, 3.0]]))
```

```
sklearn nearest neighbour index [1] = point B
sklearn distance 1.414214    by hand sqrt(2) = 1.414214
sklearn prediction 0
```

The same thing in code

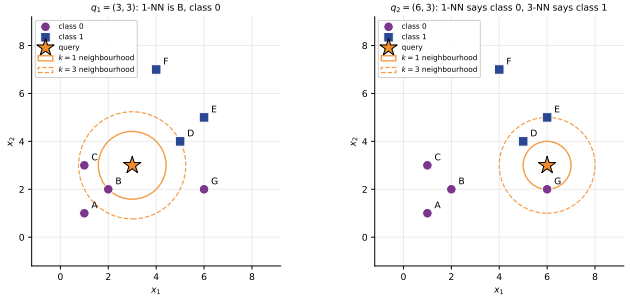
```
Xh = np.array([[1,1],[2,2],[1,3],[5,4],[6,5],[4,7],[6,2]], dtype=float)
yh = np.array([0, 0, 0, 1, 1, 1, 0])
knn1 = KNeighborsClassifier(n_neighbors=1).fit(Xh, yh)
dist, idx = knn1.kneighbors(np.array([[3.0, 3.0]]))
```

```
sklearn nearest neighbour index [1] = point B
sklearn distance 1.414214    by hand sqrt(2) = 1.414214
sklearn prediction 0
```

Index 1 is the second row, which is B. The distance agrees with $\sqrt{2}$ worked out on paper **to six decimal places**.

There is nothing inside `KNeighborsClassifier` that you did not just do with a pen.

Drawn: the seven points and the two queries



The circles have radius equal to the distance to the 1st and the 3rd nearest neighbour, so **what falls inside each circle is exactly what votes**. Look at $G = (6, 2)$: labelled class 0, but sitting over among the class-1 points. A mislabelled record, or a genuine oddity.

By hand: the query where k changes the answer

Classify $q_2 = (6, 3)$ — which lands right next to the odd point G.

rank	point	d^2	d	class
1	G	1	1.0000	0
2	D	2	1.4142	1
3	E	4	2.0000	1
4	B	17	4.1231	0
5	F	20	4.4721	1
6	C	25	5.0000	0
7	A	29	5.3852	0

Take the vote at four values of k

- $k = 1$: $\{G\}$. Votes 0:1, 1:0 $\rightarrow$ **0**
- $k = 3$: $\{G, D, E\}$. Votes 0:1, 1:2 $\rightarrow$ **1**
- $k = 5$: $\{G, D, E, B, F\}$. Votes 0:2, 1:3 $\rightarrow$ **1**
- $k = 7$: everything. Votes 0:4, 1:3 $\rightarrow$ **0**

Same data, same query, same metric — four values of k, and the answer changes twice.

Predict first #2

Those four hand-computed votes, run through the library. **Before the reveal: what does $k = 7$ give, and why is it not a coincidence?**

Predict first #2

Those four hand-computed votes, run through the library. **Before the reveal: what does $k = 7$ give, and why is it not a coincidence?**

```
k=1  votes  class0=1  class1=0  ->  predict 0   (sklearn 0)
k=3  votes  class0=1  class1=2  ->  predict 1   (sklearn 1)
k=5  votes  class0=2  class1=3  ->  predict 1   (sklearn 1)
k=7  votes  class0=4  class1=3  ->  predict 0   (sklearn 0)
```

Predict first #2

Those four hand-computed votes, run through the library. **Before the reveal: what does $k = 7$ give, and why is it not a coincidence?**

```
k=1  votes  class0=1  class1=0  ->  predict 0   (sklearn 0)
k=3  votes  class0=1  class1=2  ->  predict 1   (sklearn 1)
k=5  votes  class0=2  class1=3  ->  predict 1   (sklearn 1)
k=7  votes  class0=4  class1=3  ->  predict 0   (sklearn 0)
```

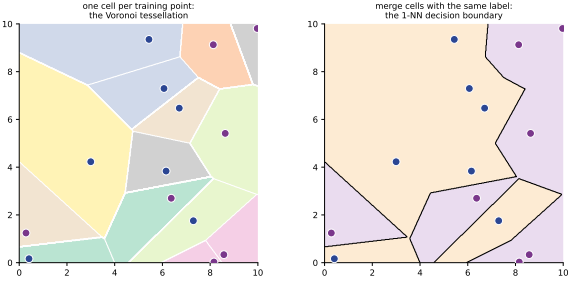
Three lessons in one table

At $k = 1$ a single odd training point dictates the answer. At $k = 3$ and $k = 5$ the vote overrules it — *that is what averaging is for*.

And at $k = 7 = n$?

The classifier has stopped looking at q_2 at all and returns the commonest label in the data. **At $k = n$, k-NN is the majority-class baseline.** Remember that; we cash it in twice today.

1-NN is a Voronoi tessellation



Each training point owns everything closer to it than to any other point — its **Voronoi cell**. Colour each cell by its point's label, erase the internal walls, and what remains is the 1-NN decision boundary. It is jagged because it is made of *perpendicular bisectors between individual data points*: **move one training point and the boundary moves.**

$k = 1$ memorises — and here is the proof

Ask 1-NN about a row that is *in* the training set. The nearest training point to that row is the row itself, at distance 0, and its label is the true label.

```
nearest neighbour of the first five training rows:
  row 0 -> training row 0 at distance 0.000000
  row 1 -> training row 1 at distance 0.000000
  row 2 -> training row 2 at distance 0.000000
  row 3 -> training row 3 at distance 0.000000
  row 4 -> training row 4 at distance 0.000000
every training row is its own nearest neighbour: True
training accuracy at k=1: 1.000000
```

$k = 1$ memorises — and here is the proof

Ask 1-NN about a row that is *in* the training set. The nearest training point to that row is the row itself, at distance 0, and its label is the true label.

```
nearest neighbour of the first five training rows:
  row 0 -> training row 0 at distance 0.000000
  row 1 -> training row 1 at distance 0.000000
  row 2 -> training row 2 at distance 0.000000
  row 3 -> training row 3 at distance 0.000000
  row 4 -> training row 4 at distance 0.000000
every training row is its own nearest neighbour: True
training accuracy at k=1: 1.000000
```

"My model got 100% accuracy"

Every year somebody reports a perfect k-NN score and is delighted. **A training accuracy of** 1.0 **at $k = 1$ is a consequence of arithmetic, not of learning**, it happens on every dataset including one with random labels, and it is entirely compatible with the model being useless. Quote a cross-validated or held-out score. Always.

The gap between training and test accuracy

Standardised `breast_cancer`, a 70/30 stratified split.

k	train acc	test acc	gap
1	1.0000	0.9474	+0.0526
3	0.9899	0.9415	+0.0484
5	0.9799	0.9474	+0.0325
11	0.9774	0.9357	+0.0417
25	0.9598	0.9415	+0.0183
51	0.9523	0.9474	+0.0049
101	0.9271	0.9181	+0.0090
199	0.8643	0.8538	+0.0105

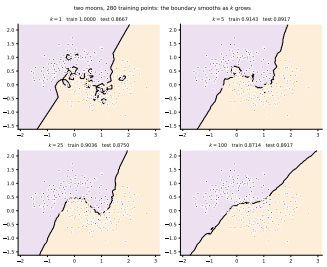
The gap between training and test accuracy

Standardised `breast_cancer`, a 70/30 stratified split.

k	train acc	test acc	gap
1	1.0000	0.9474	+0.0526
3	0.9899	0.9415	+0.0484
5	0.9799	0.9474	+0.0325
11	0.9774	0.9357	+0.0417
25	0.9598	0.9415	+0.0183
51	0.9523	0.9474	+0.0049
101	0.9271	0.9181	+0.0090
199	0.8643	0.8538	+0.0105

The gap column shrinks as k grows: $+0.0526$ at $k = 1$ down to $+0.0049$ at $k = 51$. A large train-test gap is the signature of **over-fitting**, and in k-NN over-fitting is controlled *entirely* by k. **Note also that the test column barely moves.**

Drawn: what k does to the boundary



Small k: low bias, high variance

Few assumptions; fits any shape, including the shape of the noise. Training accuracy 1.0000 at $k = 1$.

Large k: high bias, low variance

A smooth, stable boundary that may be too smooth. At $k = n$ it is the baseline. Effective parameters $\approx n/k$.

Test accuracy across these four panels varies by about one and a half points. The boundary changes far more dramatically than the score does.

Predict first #3

Standardised `breast_cancer` inside a `Pipeline`, five-fold cross-validation, k from 1 to 398 — which is n minus the held-out fold. **Which k wins, and what exact number comes out at $k = 398$?**

Predict first #3

Standardised `breast_cancer` inside a `Pipeline`, five-fold cross-validation, k from 1 to 398 — which is n minus the held-out fold. **Which k wins, and what exact number comes out at $k = 398$?**

```
   k    5-fold CV accuracy      std
   1              0.9578        0.0140
   2              0.9491        0.0217
   3              0.9649        0.0157
   5              0.9649        0.0215
   7              0.9649        0.0192
   9              0.9649        0.0215
  11              0.9666        0.0129
  15              0.9613        0.0106
  21              0.9578        0.0140
  31              0.9543        0.0128
  45              0.9473        0.0123
  65              0.9438        0.0142
  95              0.9297        0.0146
 135              0.9156        0.0204
 199              0.8805        0.0211
 285              0.8015        0.0229
 398              0.6274        0.0039
best k = 11 at CV accuracy 0.9666
majority-class baseline 0.6274
```

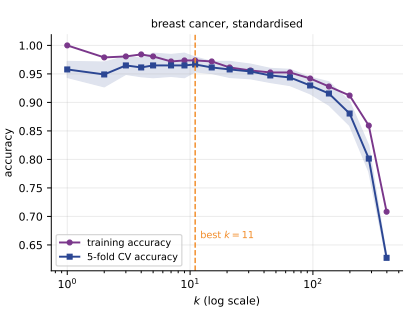
Predict first #3

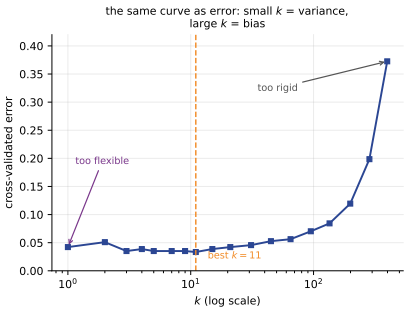
Standardised `breast_cancer` inside a `Pipeline`, five-fold cross-validation, k from 1 to 398 — which is n minus the held-out fold. **Which k wins, and what exact number comes out at $k = 398$?**

```
   k    5-fold CV accuracy      std
   1              0.9578          0.0140
   2              0.9491          0.0217
   3              0.9649          0.0157
   5              0.9649          0.0215
   7              0.9649          0.0192
   9              0.9649          0.0215
  11              0.9666          0.0129
  15              0.9613          0.0106
  21              0.9578          0.0140
  31              0.9543          0.0128
  45              0.9473          0.0123
  65              0.9438          0.0142
  95              0.9297          0.0146
 135              0.9156          0.0204
 199              0.8805          0.0211
 285              0.8015          0.0229
 398              0.6274          0.0039
best k = 11 at CV accuracy 0.9666
majority-class baseline 0.6274
```

0.6274 **was predictable before we ran anything** — at $k = n$ the classifier returns the commonest label, so its accuracy *is* the majority-class proportion.

Drawn: the validation curve is U-shaped





Read the right-hand end for free

The accuracy at $k = n$ *is* the class balance. A free diag… end…

And do not over-read the peak

$k = 3$ to $k = 15$ are all within one standard …

By hand: two patients from `breast_cancer`

k-NN's only input is a distance, and a distance is a sum over columns. Take rows 0 and 1, and look at two of their thirty columns.

feature	patient 0	patient 1	difference	$(\text{difference})^2$
mean area	1001.0000	1326.0000	−325.0000	105625.0000
mean smoothness	0.1184	0.0847	0.0337	0.0011

The squared distance over *all thirty* columns is 116779.5720. So *mean area* alone supplies $105625/116779.5720 = \mathbf{90.45}\%$ of it, and *mean smoothness* supplies about $\mathbf{0.000001}\%$ — one part in a hundred million.

Both are real medical measurements. Neither is more important than the other. **The only difference between them is the unit they happen to be recorded in.**

The same split, computed over all thirty columns

```
p, q = bc.data[0], bc.data[1]
tot = ((p - q) ** 2).sum()
share = (p - q) ** 2 / tot
```

```
squared distance over all 30 features 116779.5720
  mean area                contributes  90.4482%
  area error               contributes   5.3876%
  worst area               contributes   3.3987%
  worst perimeter          contributes   0.5700%
the two area columns together contribute 93.8469%
the 27 columns that are not areas contribute 0.7655%
```

The same split, computed over all thirty columns

```
p, q = bc.data[0], bc.data[1]
tot = ((p - q) ** 2).sum()
share = (p - q) ** 2 / tot
```

```
squared distance over all 30 features 116779.5720
  mean area                contributes  90.4482%
  area error               contributes   5.3876%
  worst area               contributes   3.3987%
  worst perimeter          contributes   0.5700%
the two area columns together contribute 93.8469%
the 27 columns that are not areas contribute 0.7655%
```

Three columns supply 99.23% of the distance

The other twenty-seven measurements share **0.7655%** between them. **Unscaled k-NN on this dataset is a classifier that uses three columns and ignores the rest** — and nothing in its output says so.

It changes which point is nearest

```
raw1 = KNeighborsClassifier(1).fit(Xbtr, ybtr) # unscaled
std1 = KNeighborsClassifier(1).fit(Xbtr_s, ybtr) # standardised
n_raw = raw1.kneighbors(Xbte, return_distance=False)[:, 0]
n_std = std1.kneighbors(Xbte_s, return_distance=False)[:, 0]
```

```
test rows whose nearest neighbour changes when we standardise: 164 of 171
of those, 19 change the predicted class
example: test row 5, true label 1
  raw          nearest is training row 206, label 0
  standardised nearest is training row 69, label 1
```

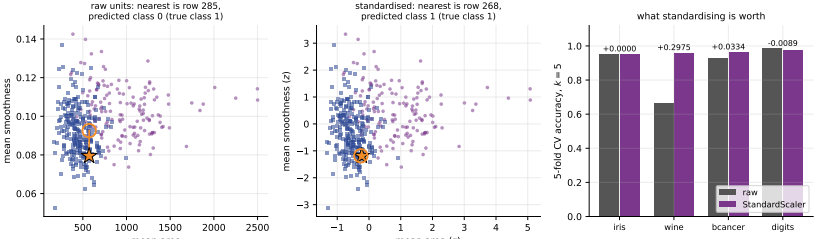
It changes which point is nearest

```
raw1 = KNeighborsClassifier(1).fit(Xbtr, ybtr) # unscaled
std1 = KNeighborsClassifier(1).fit(Xbtr_s, ybtr) # standardised
n_raw = raw1.kneighbors(Xbte, return_distance=False)[:, 0]
n_std = std1.kneighbors(Xbte_s, return_distance=False)[:, 0]
```

```
test rows whose nearest neighbour changes when we standardise: 164 of 171
of those, 19 change the predicted class
example: test row 5, true label 1
  raw          nearest is training row 206, label 0
  standardised nearest is training row 69, label 1
```

164 of 171 held-out patients get a *different* nearest neighbour, and for 19 of them the **answer** changes. Test row 5 is genuinely class 1; standardising is what gets it right.

Drawn: the neighbour flips



Left and centre: one held-out patient (star) and its nearest training neighbour (circled), on two features, before and after standardising. The neighbour changes and so does the predicted class. Right: what standardising is worth to a $k = 5$ classifier on four bundled datasets — which is the next slide's question.

Predict first #4

`KNeighborsClassifier(5)`, five-fold cross-validation, raw against a `StandardScaler` pipeline, on four bundled datasets.

Rank the four gains. Which is biggest — and is any of them negative?

Predict first #4

`KNeighborsClassifier(5)`, five-fold cross-validation, raw against a `StandardScaler` pipeline, on four bundled datasets.

Rank the four gains. Which is biggest — and is any of them negative?

```
dataset                raw     scaled      gain
iris                0.9533     0.9533   +0.0000
wine                0.6633     0.9608   +0.2975
breast_cancer       0.9315     0.9649   +0.0334
digits              0.9855     0.9766   -0.0089
feature std ratio (widest/narrowest column):
  iris                  4.1
  wine               2530.3
  breast_cancer    215170.7
```

Predict first #4

`KNeighborsClassifier(5)`, five-fold cross-validation, raw against a `StandardScaler` pipeline, on four bundled datasets.

Rank the four gains. Which is biggest — and is any of them negative?

```
dataset                 raw     scaled      gain
iris                 0.9533     0.9533   +0.0000
wine                 0.6633     0.9608   +0.2975
breast_cancer        0.9315     0.9649   +0.0334
digits               0.9855     0.9766   −0.0089
feature std ratio (widest/narrowest column):
  iris                   4.1
  wine                2530.3
  breast_cancer     215170.7
```

The size of the scale disparity does *not* predict the size of the gain

`wine` +0.2975 at a ratio of 2530; `breast_cancer` only +0.0334 at a ratio of 215,171 — because there the dominant area columns happen also to be genuinely informative. `iris`: nothing to fix, all four columns are centimetres. `digits`: −0.0089 — **scaling made it worse**, because dividing an almost-always-zero border pixel by its tiny standard deviation inflates its noise into a full-sized feature.

So: when to scale, and where to put the scaler

The rule

Scale when the columns are in different units. Do not scale reflexively when they are already commensurable: standardising a near-constant column amplifies its noise. On `digits` that cost 0.0089 of accuracy.

Fit the scaler on the training rows only

`StandardScaler` learns a mean and a standard deviation. Those are *numbers learned from data*, so the preprocessing rule applies unchanged. In practice: put the scaler and the classifier in a `Pipeline` and let `cross_val_score` refit both inside every fold.

Thirty points of accuracy on `wine` for four lines of preprocessing. **No modelling decision in this lecture is worth anything like that much.**

The Minkowski family

$$d_p(a, b) = \left(\sum_{j=1}^{d} |a_j - b_j|^p \right)^{1/p}, \qquad p \geq 1$$

p	name	character
1	Manhattan, city-block, L_1	sum of the coordinate differences; robust to one large discrepancy
2	Euclidean, L_2	straight-line distance; the default
p	Minkowski-p	`KNeighborsClassifier(p=...)`
∞	Chebyshev, L_∞	the single largest coordinate difference

A diamond for $p = 1$, a circle for $p = 2$, a square for $p = \infty$. **Changing p changes the shape of "nearby", and therefore changes who your neighbours are.**

By hand: where two metrics disagree

Query $q = (0, 0)$. Two candidates: $P = (3, 0)$ of class 0, and $Q = (2, 2)$ of class 1.

Manhattan, $p = 1$

$d_1(q, P) = 3 + 0 = 3$
$d_1(q, Q) = 2 + 2 = 4$
Nearest is $P \Rightarrow$ **class 0**

Euclidean, $p = 2$

$d_2(q, P) = \sqrt{9} = 3$
$d_2(q, Q) = \sqrt{8} = 2.8284$
Nearest is $Q \Rightarrow$ **class 1**

Where does the switch happen?

Set the two Minkowski distances equal:

$$3 = \left(2^p + 2^p\right)^{1/p} = 2^{1+1/p}.$$

Take $\log_2$: $\log_2 3 = 1 + 1/p$, so

$$p^* = \frac{1}{\log_2 3 - 1} = 1.709511.$$

Below p^*: class 0. Above it: class 1.

Same data, same k, opposite answers.

The crossover, in code

```
p=1          d(q,P)=3.0000   d(q,Q)=4.0000   nearest = P
p=1.5        d(q,P)=3.0000   d(q,Q)=3.1748   nearest = P
p=1.7095     d(q,P)=3.0000   d(q,Q)=3.0000   nearest = P
p=2          d(q,P)=3.0000   d(q,Q)=2.8284   nearest = Q
p=3          d(q,P)=3.0000   d(q,Q)=2.5198   nearest = Q
p=10         d(q,P)=3.0000   d(q,Q)=2.1435   nearest = Q
crossover: 3 = 2^(1+1/p)  gives  p = 1/(log2(3)-1) = 1.709511
Chebyshev (p -> inf): d(q,P)=3.0   d(q,Q)=2.0
```

The crossover, in code

```
p=1          d(q,P)=3.0000   d(q,Q)=4.0000   nearest = P
p=1.5        d(q,P)=3.0000   d(q,Q)=3.1748   nearest = P
p=1.7095     d(q,P)=3.0000   d(q,Q)=3.0000   nearest = P
p=2          d(q,P)=3.0000   d(q,Q)=2.8284   nearest = Q
p=3          d(q,P)=3.0000   d(q,Q)=2.5198   nearest = Q
p=10         d(q,P)=3.0000   d(q,Q)=2.1435   nearest = Q
crossover: 3 = 2^(1+1/p)  gives  p = 1/(log2(3)-1) = 1.709511
Chebyshev (p -> inf): d(q,P)=3.0   d(q,Q)=2.0
```

```
p=1 (manhattan) predicts class 0 for the origin
p=2 (euclidean) predicts class 1 for the origin
```

The crossover, in code

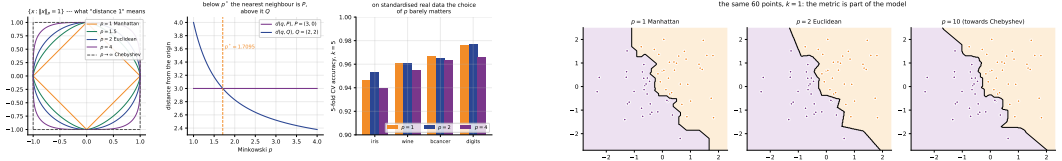
```
p=1          d(q,P)=3.0000   d(q,Q)=4.0000   nearest = P
p=1.5        d(q,P)=3.0000   d(q,Q)=3.1748   nearest = P
p=1.7095     d(q,P)=3.0000   d(q,Q)=3.0000   nearest = P
p=2          d(q,P)=3.0000   d(q,Q)=2.8284   nearest = Q
p=3          d(q,P)=3.0000   d(q,Q)=2.5198   nearest = Q
p=10         d(q,P)=3.0000   d(q,Q)=2.1435   nearest = Q
crossover: 3 = 2^(1+1/p)  gives  p = 1/(log2(3)-1) = 1.709511
Chebyshev (p -> inf): d(q,P)=3.0   d(q,Q)=2.0
```

```
p=1 (manhattan) predicts class 0 for the origin
p=2 (euclidean) predicts class 1 for the origin
```

The metric is part of the model, not an implementation detail

Two k-NN classifiers with the *same* k and the *same* data are **different classifiers** if their p differs.

Drawn: unit balls, the crossover, and three boundaries



Left group: the unit balls, the $p^* = 1.7095$ crossover, and the (small) effect on real data. Right group: the same sixty points at the same $k = 1$ under three metrics — Manhattan builds diagonal facets, Euclidean builds perpendicular bisectors, near-Chebyshev builds axis-aligned steps.

Does p matter in practice? Honestly: not much

Four standardised datasets, $k = 5$, four metrics.

dataset	p=1	p=2	p=4	chebyshev
iris	0.9467	0.9533	0.9400	0.9400
wine	0.9606	0.9608	0.9549	0.9214
breast_cancer	0.9666	0.9649	0.9631	0.9420
digits	0.9761	0.9766	0.9655	0.9255

Does p matter in practice? Honestly: not much

Four standardised datasets, $k = 5$, four metrics.

dataset	p=1	p=2	p=4	chebyshev
iris	0.9467	0.9533	0.9400	0.9400
wine	0.9606	0.9608	0.9549	0.9214
breast_cancer	0.9666	0.9649	0.9631	0.9420
digits	0.9761	0.9766	0.9655	0.9255

$p = 1$ and $p = 2$ are within 0.002 of each other *everywhere*. Chebyshev is consistently the worst, by up to 0.05 on `digits`, because throwing away all but the single largest coordinate difference discards most of the information.

Treat p as a hyperparameter worth one line in a grid search — **not as the place where your accuracy is hiding**. Compare with the +0.2975 that scaling was worth.

By hand: distance-weighted voting

A plain vote treats the nearest neighbour and the kth-nearest as equals. `weights='distance'` gives each neighbour weight $1/d$.

neighbour	n_1	n_2	n_3	n_4	n_5
distance d	1	2	4	4	5
label	1	0	0	0	0
weight $1/d$	1.0000	0.5000	0.2500	0.2500	0.2000

Uniform vote

Class 0 gets four, class 1 gets one. **Predict 0** — comfortably.

Weighted vote

Class 1: 1.0000. Class 0: $0.5 + 0.25 + 0.25 + 0.2 = 1.2000$. **Predict 0** — but only just.

The margin collapsed from 4:1 to 1.2:1.0. **Move n_1 to $d = 0.8$ and its weight becomes 1.25: the weighted vote flips while the uniform vote is unchanged.**

Weighted voting: arithmetic, then measurement

```
neighbour distances [1. 2. 4. 4. 5.]
neighbour labels    [1 0 0 0 0]
uniform vote : class1 = 1, class0 = 4  ->  predict 0
weights 1/d         [1.   0.5  0.25 0.25 0.2 ]
distance vote: class1 = 1.0000, class0 = 1.2000  ->  predict 0
```

Weighted voting: arithmetic, then measurement

```
neighbour distances [1. 2. 4. 4. 5.]
neighbour labels    [1 0 0 0 0]
uniform vote : class1 = 1, class0 = 4  ->  predict 0
weights 1/d         [1.   0.5  0.25 0.25 0.2 ]
distance vote: class1 = 1.0000, class0 = 1.2000  ->  predict 0
```

```
dataset             uniform   distance     gain
iris                 0.9533     0.9533  +0.0000
wine                 0.9665     0.9608  -0.0057
breast_cancer        0.9613     0.9649  +0.0035
digits               0.9622     0.9627  +0.0006
training accuracy of k=25 with distance weights: 1.0000
```

Weighted voting: arithmetic, then measurement

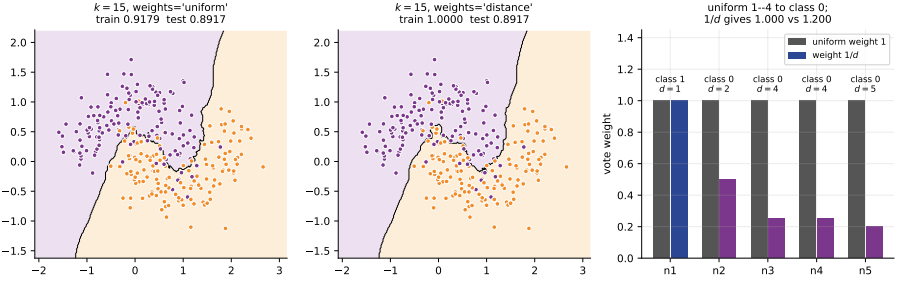
```
neighbour distances [1. 2. 4. 4. 5.]
neighbour labels    [1 0 0 0 0]
uniform vote : class1 = 1, class0 = 4  ->  predict 0
weights 1/d         [1.   0.5  0.25 0.25 0.2 ]
distance vote: class1 = 1.0000, class0 = 1.2000  ->  predict 0
```

```
dataset             uniform   distance     gain
iris                 0.9533     0.9533  +0.0000
wine                 0.9665     0.9608  -0.0057
breast_cancer        0.9613     0.9649  +0.0035
digits               0.9622     0.9627  +0.0006
training accuracy of k=25 with distance weights: 1.0000
```

Distance weighting makes *every* k memorise

Worth between -0.0057 and $+0.0035$ here — not a free win. Its real value: it never produces a tie and it degrades gracefully when k is too large. But read the last line: at $k = 25$ it **still scores** 1.0000 **on its own training set**, because a training point is at distance 0 from itself and $1/0$ outvotes everything.

Drawn: uniform against weighted at $k = 15$



Left and centre: the same $k = 15$, uniform and distance-weighted. The weighted version reaches a training accuracy of 1.0000 *even at* $k = 15$. Right: the five-neighbour hand calculation, drawn.

Predict first #5

500 points drawn uniformly at random in $[0, 1]^d$, one query, averaged over 20 repetitions.
k-NN rests on one assumption: that *near* means something.

At $d = 1000$, how many times farther away is the farthest point than the nearest?

Predict first #5

500 points drawn uniformly at random in $[0,1]^d$, one query, averaged over 20 repetitions.
k-NN rests on one assumption: that *near* means something.

At $d = 1000$, how many times farther away is the farthest point than the nearest?

d	mean d_min	mean d_max	d_max/d_min	(max-min)/min
1	0.0007	0.7705	1156.2859	3577.0046
2	0.0177	0.9949	56.3273	85.5474
5	0.1913	1.4461	7.5594	7.0526
10	0.5743	1.9077	3.3218	2.3636
20	1.0921	2.4688	2.2605	1.2790
50	2.1574	3.4987	1.6217	0.6240
100	3.3890	4.7604	1.4046	0.4053
500	8.4577	9.7935	1.1579	0.1580
1000	12.2313	13.6006	1.1119	0.1120

Predict first #5

500 points drawn uniformly at random in $[0,1]^d$, one query, averaged over 20 repetitions. k-NN rests on one assumption: that *near* means something.

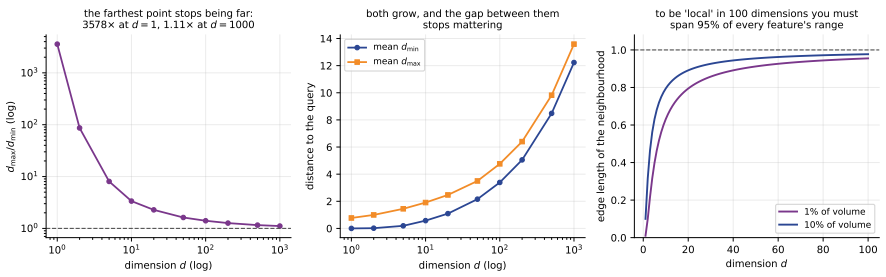
At $d = 1000$, how many times farther away is the farthest point than the nearest?

d	mean d_min	mean d_max	d_max/d_min	(max-min)/min
1	0.0007	0.7705	1156.2859	3577.0046
2	0.0177	0.9949	56.3273	85.5474
5	0.1913	1.4461	7.5594	7.0526
10	0.5743	1.9077	3.3218	2.3636
20	1.0921	2.4688	2.2605	1.2790
50	2.1574	3.4987	1.6217	0.6240
100	3.3890	4.7604	1.4046	0.4053
500	8.4577	9.7935	1.1579	0.1580
1000	12.2313	13.6006	1.1119	0.1120

1.11×

In one dimension the farthest point is over *a thousand times* as far away as the nearest. In a thousand dimensions it is 1.11 times as far. **The nearest neighbour is barely nearer than the farthest, and a classifier whose whole mechanism is "find the nearest" has nothing left to work with.**

Drawn: distances concentrate



Left: the ratio $d_{\max}/d_{\min}$ collapses towards 1. Centre: both distances *grow*, and the gap between them stops being a meaningful fraction of either. Right: the edge length a neighbourhood must have to enclose a fixed fraction of the volume — which is the next slide.

By hand: how big is a "small" neighbourhood?

Data fill the unit cube $[0,1]^d$ uniformly. You want a cubical neighbourhood holding 1% of the points. Its volume is e^d, so $e^d = 0.01$, that is $\mathbf{e = 0.01^{1/d}}$.

```
to capture 1% of the volume of the unit cube, a neighbourhood
must span this fraction of the range of every feature:
  d=   1   0.01^(1/d) = 0.0100
  d=   2   0.01^(1/d) = 0.1000
  d=   5   0.01^(1/d) = 0.3981
  d=  10   0.01^(1/d) = 0.6310
  d=  20   0.01^(1/d) = 0.7943
  d=  50   0.01^(1/d) = 0.9120
  d= 100   0.01^(1/d) = 0.9550
```

By hand: how big is a "small" neighbourhood?

Data fill the unit cube $[0,1]^d$ uniformly. You want a cubical neighbourhood holding 1% of the points. Its volume is e^d, so $e^d = 0.01$, that is $\mathbf{e = 0.01^{1/d}}$.

```
to capture 1% of the volume of the unit cube, a neighbourhood
must span this fraction of the range of every feature:
  d=   1   0.01^(1/d) = 0.0100
  d=   2   0.01^(1/d) = 0.1000
  d=   5   0.01^(1/d) = 0.3981
  d=  10   0.01^(1/d) = 0.6310
  d=  20   0.01^(1/d) = 0.7943
  d=  50   0.01^(1/d) = 0.9120
  d= 100   0.01^(1/d) = 0.9550
```

In 100 dimensions that is a box spanning 95.5% of *every single feature*

It is not a neighbourhood; it is nearly the whole dataset. **The word "local" has no meaning in high dimensions** — and k-NN is a purely local method. Reproduce this argument in the examination; it is four lines of arithmetic.

The version that will actually happen to you

Take `iris`, whose four measurements do a fine job, and bolt on columns of *pure noise*.

noise cols	KNN(5)	tree	logistic
0	0.9533	0.9333	0.9600
2	0.8800	0.9333	0.9400
5	0.8333	0.9333	0.9667
10	0.8267	0.9200	0.9400
25	0.8133	0.9267	0.9333
50	0.6867	0.9000	0.7600
100	0.5333	0.9200	0.6800
200	0.4667	0.9133	0.7000
400	0.5200	0.9067	0.6067

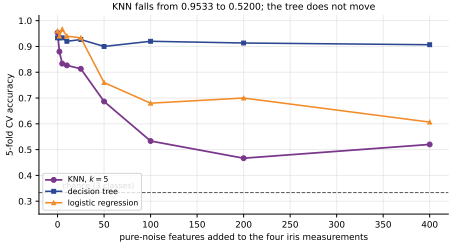
The version that will actually happen to you

Take `iris`, whose four measurements do a fine job, and bolt on columns of *pure noise*.

noise cols	KNN(5)	tree	logistic
0	0.9533	0.9333	0.9600
2	0.8800	0.9333	0.9400
5	0.8333	0.9333	0.9667
10	0.8267	0.9200	0.9400
25	0.8133	0.9267	0.9333
50	0.6867	0.9000	0.7600
100	0.5333	0.9200	0.6800
200	0.4667	0.9133	0.7000
400	0.5200	0.9067	0.6067

k-NN falls from 0.9533 to **0.5200**, through a low of 0.4667 — about half its accuracy gone, closing on the one-in-three chance level. *Two* noise columns already cost it seven points. The decision tree ends at 0.9067, essentially where it started.

Drawn: why the tree survives and k-NN does not



k-NN has no feature selection built in

A decision tree *chooses* which feature to split on and can ignore the useless ones. k-NN cannot: **every feature enters the distance sum with equal weight, whether or not it carries any information**. So before running k-NN on a wide table, select features, or reduce with PCA, or use a model that selects for you. **Adding a column costs k-NN accuracy unless that column earns its place.**

Ties, and how scikit-learn breaks them

Four points at the corners of a square, labels $0, 1, 1, 0$, query at the exact centre.

```
all four neighbours at distance [1.4142 1.4142 1.4142 1.4142]
their labels [0 1 1 0]
k=4 probabilities [0.5 0.5] -> predict 0
k=2 probabilities [0.5 0.5] -> predict 0
k=3 probabilities [0.33333333 0.66666667] -> predict 1
```

Ties, and how scikit-learn breaks them

Four points at the corners of a square, labels $0, 1, 1, 0$, query at the exact centre.

```
all four neighbours at distance [1.4142 1.4142 1.4142 1.4142]
their labels [0 1 1 0]
k=4 probabilities [0.5 0.5] -> predict 0
k=2 probabilities [0.5 0.5] -> predict 0
k=3 probabilities [0.33333333 0.66666667] -> predict 1
```

Deterministically, towards the smaller label

It takes the `argmax` of the vote counts, which returns the *first* maximum. **It does not toss a coin and it does not warn you.**

Three consequences

(1) Use an odd k for two classes — $k = 2$ scored 0.9491, below both $k = 1$ and $k = 3$. **(2)** An odd k does *not* help with three or more classes: 3:3:3 at $k = 9$. **(3)** Distance ties are broken by row order, so reordering the training rows can change the prediction.

Imbalanced classes: the most important table today

2000 rows, a 94:6 split, ten features.

```
class counts [1882  118]
   k   accuracy   minority recall   minority predicted
   1     0.9367            0.4000                31
   3     0.9483            0.2571                14
   5     0.9550            0.2571                10
  15     0.9417            0.0000                 0
  51     0.9417            0.0000                 0
 151     0.9417            0.0000                 0
always predicting the majority class scores 0.9417
k=51 with distance weights: accuracy 0.9417, minority recall 0.0000
```

Imbalanced classes: the most important table today

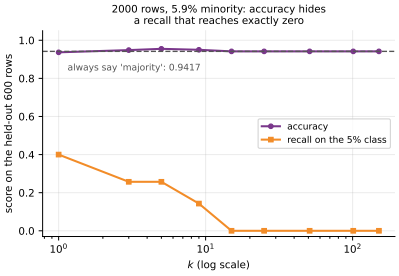
2000 rows, a 94:6 split, ten features.

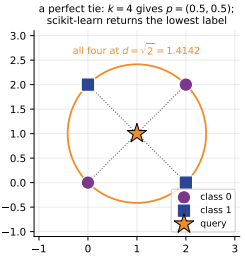
```
class counts [1882  118]
   k   accuracy   minority recall   minority predicted
   1     0.9367            0.4000                31
   3     0.9483            0.2571                14
   5     0.9550            0.2571                10
  15     0.9417            0.0000                 0
  51     0.9417            0.0000                 0
 151     0.9417            0.0000                 0
always predicting the majority class scores 0.9417
k=51 with distance weights: accuracy 0.9417, minority recall 0.0000
```

0.9417 and 0.0000 describe the same classifier

At $k \geq 15$ it predicts the minority class **zero times** on 600 held-out rows and scores exactly the baseline. The mechanism is arithmetic: at 6% minority, a neighbourhood of 51 contains about 3 minority points and needs 26 to win. **Never report accuracy alone on imbalanced data.**

Drawn: accuracy hides it completely





Left: accuracy stays around 0.94 for every k while minority recall falls to *exactly zero*. Right: the four-way distance tie. Three fixes, in order of preference: **resample** the training set; use `predict_proba` with a threshold below 0.5; or choose an algorithm supporting `class_weight='balanced'` — which `KNeighborsClassifier`, note, does *not*.

The complete failure list — and why we teach it anyway

Failure	Symptom	Fix
unscaled features	one column supplies 90% of the distance	`StandardScaler` in a `Pipeline`
k too small	training 1.0, test much lower	raise k; cross-validate
k too large	accuracy equals the baseline	lower k
even k, two classes	silent tie-breaking	odd k, or `weights='distance'`
irrelevant features	accuracy falls as columns are added	feature selection, or PCA
high dimension	all distances nearly equal	reduce d; another algorithm
imbalanced classes	minority recall 0, accuracy high	resample; move the threshold
slow prediction	latency grows with n	index in low d; an eager learner
large model	the training data ships with the model	an eager learner
missing values	`nan` poisons every distance	impute before fitting
categorical features	"distance" between two words	encode, or a mixed metric

Four reasons it survives

1. **The honest baseline.** One hyperparameter, no assumptions. If your elaborate model cannot beat it, it is not earning its keep.
2. **No assumption about the boundary.** Cover–Hart: asymptotically 1-NN's error is at most twice the best possible.
3. **Naturally multi-class.** On `digits` it scored 0.9796 against logistic regression's 0.9611.
4. **It needs no feature vector** — only a distance. Edit distance, dynamic time warping, graph distance.

Summary — fourteen lines

1. Classification predicts a label from a finite set. Han, Kamber and Pei's general approach has two steps: **model construction**, then **model usage** with accuracy estimated on an independent test set.
2. Read the confusion matrix, not the accuracy. One 70/30 split of iris scored 1.0000 and said nothing; cross-validated over all 150 rows the score was 0.9600 and *every* error was between *versicolor* and *virginica*.
3. The spread between five real classifiers on breast_cancer (0.9262 to 0.9789) is smaller than the gap from the worst of them to the 0.6274 baseline.
4. **Eager** works at fit time; **lazy** works at predict time. On `digits` k-NN does $125.7\times$ the arithmetic per prediction; the predict/fit ratio is above 1 for k-NN and 0.0001 for logistic regression — a factor of 10^5.
5. A fitted k-NN *is* the data: 643,584 bytes against 5,200, and $40\times$ the rows is exactly $40\times$ the distances.
6. The algorithm: distance to every point, keep the k smallest, vote. By hand, 1-NN put $q_1 = (3, 3)$ in class 0 via B at $d = \sqrt{2}$ — and the library returned 1.414214.
7. k changes the answer: on $q_2 = (6, 3)$, $k = 1 \to 0$, $k = 3 \to 1$, $k = 5 \to 1$, $k = 7 \to 0$. **At $k = n$ it is the majority-class baseline.**
8. $k = 1$ **scores exactly** 1.000000 **on its own training set**, because every row is its own nearest neighbour at distance 0. Training accuracy is worthless evidence here.
9. The validation curve peaked at $k = 11$ with 0.9666 and fell to 0.6274 at $k = 398$. The peak is **shallow**: $k = 3$ to $k = 15$ are indistinguishable.
10. **Scaling.** *Mean area* alone supplied 90.4482% of one squared distance; the 27 non-area columns shared 0.7655%. Standardising changed the nearest neighbour of 164 of 171 test patients and the class of 19.
11. Gains at $k = 5$: `wine` $+0.2975$, `breast_cancer` $+0.0334$, `iris` $+0.0000$, `digits` $-\mathbf{0.0089}$. Scale when the units differ, not reflexively.
12. Minkowski: P wins under L_1, Q under L_2, crossover at $p^* = 1.7095$. On real data p moves accuracy by under 0.01; distance weighting by between -0.0057 and $+0.0035$ — and it makes *every* k score 1.0 on the training set.
13. **The curse.** $d_{\max}/d_{\min}$ is 1156 at $d = 1$ and 1.11 at $d = 1000$. A box holding 1% of the data in 100 dimensions spans 95.5% of every feature. Adding noise columns to iris took k-NN from 0.9533 to 0.5200; the tree went 0.9333 to 0.9067.
14. On a 94:6 problem, $k \geq 15$ predicted the minority class zero times in 600 rows while scoring 0.9417 — exactly the baseline. **Never report accuracy alone.**

Before the next lecture

Read

`notes.pdf` — the full development, the Voronoi argument, the complete failure table, and **10 practice questions with worked solutions**.

Do

Questions 1, 2 and 5, **with a pen**. Question 1 is a distance table and three values of k; question 5 is a Minkowski crossover you must derive, not guess.

Next: decision trees

Attribute selection measures, information gain, ID3, and turning a tree into rules. Carry one contrast across: k-**NN weights every column equally; a tree chooses.** That single sentence explains the last figure of this lecture.

And a habit to start now

Every k-NN you write goes in a `Pipeline` with a scaler, and every score you quote is cross-validated. If you catch yourself reporting `score(Xtr, ytr)`, stop.

Materials: `https://github.com/tali7c/Applied-Machine-Learning`

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.